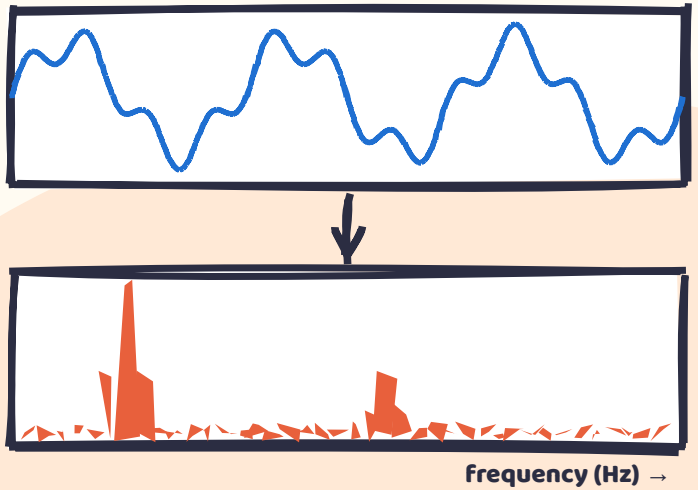
Tech Leaders · Class 24

Noise Pollution Analyser with FFT

Hear it as numbers. See it as frequencies.

Decibels | Sampling | FFT | Survey



Did you know?
Apps like Shazam identify a song in seconds by turning sound into a spectrogram with the FFT and matching its peaks.

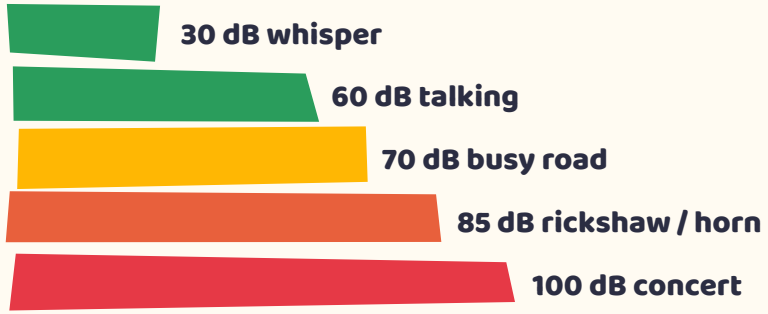
LEARN

Let's think...

How loud is too loud?

Horns, rickshaws, generators

Our cities are loud: pressure horns, rickshaws, generators during load-shedding, wedding speakers. Long exposure to loud noise damages hearing and raises stress. But 'loud' is an opinion. Can we **measure** it, and even tell **what** is making it?



It's fine!

Ears hurt!

Measure it!

Today: you build a Python **noise analyser**: record, measure level, use the **FFT** to find the main frequencies, and survey your school.

Learning objectives

By the end of this class I can...

1	**Sound + decibels:** explain **amplitude, frequency** and the **logarithmic decibel** scale, with health limits	**Proof:** I can say why 80 dB is far louder than 'twice' 40 dB
2	**Sample it:** record audio with **sounddevice** and explain **sample rate** and the **Nyquist** limit	**Proof:** my program records 3 s and plots the waveform
3	**FFT:** use NumPy's **FFT** to turn a waveform into a **spectrum** and find dominant frequencies	**Proof:** my spectrum shows the peak of a tuning-fork app at the right Hz
4	**Noise survey:** survey noise around school, **calibrate** against a phone meter and report honestly	**Proof:** our map shows levels and main noise sources at 5 spots

LEARN

Objective 3 · FFT · video

Video time!



How Shazam IDs Over 23,000 Songs Each Minute

The Wall Street Journal · about 7 minutes · youtu.be/b6xeOLjeKs0

Watch for...

1. How does Shazam turn a song into a **fingerprint**?
2. What is a **spectrogram**?
3. Why does it work even in a noisy room?

Scan to watch

After the video: If Shazam can identify songs, what could the same idea identify in a city's noise?

Objective 1 · Sound + decibels

Sound, frequency and decibels

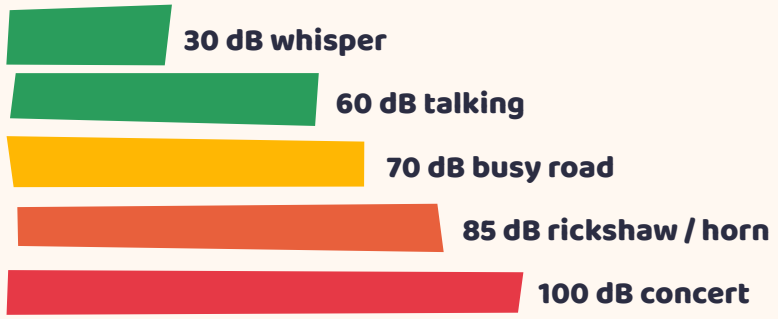
The idea

Sound is air pressure going up and down. **Frequency** (Hz) = how many times per second: low rumble ~50 Hz, voice ~100 to 3000 Hz, whistle ~2000+ Hz. **Amplitude** = how big the pressure changes are.

Our ears cover a huge range, so we use the **logarithmic** decibel scale: **+10 dB = 10× the sound power**, and sounds roughly **twice as loud**.

WHO-style guidance: long exposure above about **85 dB** risks hearing damage.

Example: the Richter scale



An earthquake of magnitude 6 isn't 'a bit more' than 5: it's far more energetic. Decibels are similar: 90 dB isn't a bit more than 80; it's 10× the power.

LEARN

Objective 2 · Sample it

Sampling: how a computer records sound

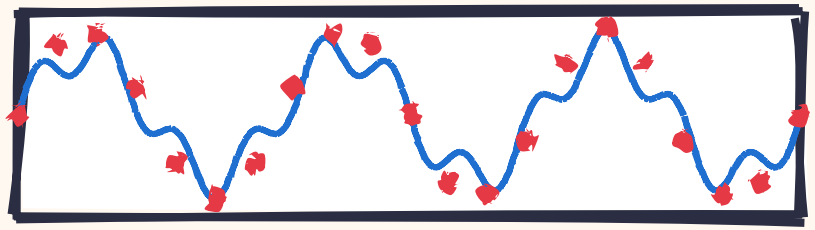
The idea

The microphone voltage is measured thousands of times per second: the **sample rate**. CD quality: **44 100** samples per second.

Nyquist limit: you can only capture frequencies up to **half** the sample rate (22 050 Hz at 44.1 kHz), which covers human hearing (~20 Hz to 20 kHz).

3 s at 44.1 kHz = **132 300** numbers in a NumPy array. Values are **relative** (−1 to +1), not true dB until **calibrated**.

Example: a flip-book of a spinning wheel



Film a fast wheel with too few frames and it looks like it spins backwards. Sample sound too slowly and high notes turn into fake low ones (aliasing).

Objective 3 · FFT

The FFT: from time to frequency

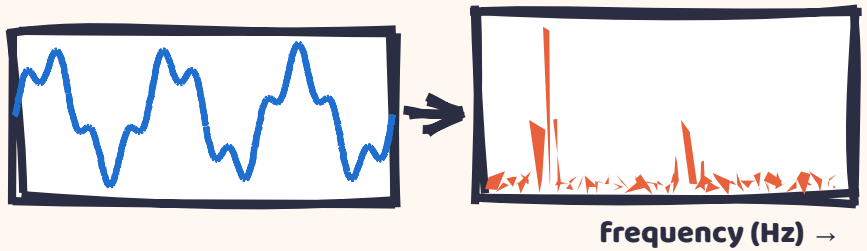
The idea

A recording is a messy wave: many frequencies added together. The **Fast Fourier Transform** splits it into a **spectrum**: how much of each frequency is present.

spectrum = np.abs(np.fft.rfft(x))
freqs = np.fft.rfftfreq(len(x), 1/fs)

The tallest peak = **dominant frequency**. A generator hums low (~50 to 150 Hz); a pressure horn has strong peaks ~400 to 3000 Hz; birds are higher. A **spectrogram** shows the spectrum changing over time.

Example: a prism splitting light



White light looks like one colour, but a prism splits it into a rainbow. The FFT is a prism for sound: it shows the hidden 'colours' (frequencies).

Objective 4 · Noise survey

Honest measurement: calibration and limits

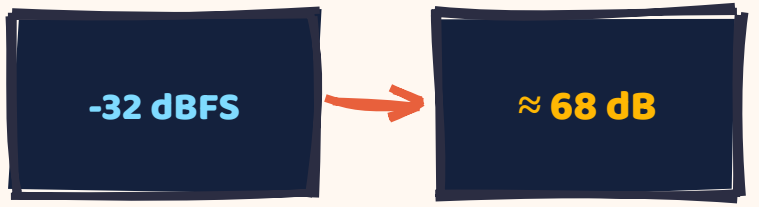
The idea

Laptop mics change their own gain and aren't measurement mics. We compute a **relative** level: **dBFS = 20 × log10(RMS)** (0 = maximum, negative below).

Calibrate: measure the same sound with a phone sound-meter app (or a real meter) and find the **offset**: dB ≈ dBFS + offset. It's approximate!

Survey method: same height, 30 s per spot, same time of day, note the main sources. Report uncertainty: '±5 dB'.

Example: a kitchen scale



A cheap scale might always read 50 g heavy. Weigh a known 1 kg bag, find the error, and correct every reading. Calibration makes cheap sensors useful.

Real world

Sound data at work



Music recognition
Spectrogram peaks identify songs (Shazam)



Machine health
A bearing about to fail changes its frequency signature



Medicine
Digital stethoscopes analyse heart and lung sounds



Noise maps
Cities map noise to plan quieter roads and schools



Where in our city should noise be measured most, and who should see the results?

SDG 3 + 11
Quieter cities mean better health and better learning. Data gives communities evidence to ask for change.

LEARN

Quick quiz

Sound maths

Question	Answer
1. 70 dB to 80 dB: how much more sound power?	
2. Sample rate 8000 Hz. Highest frequency captured?	
3. What does the FFT output?	
4. Generator hum: low or high frequency?	
5. Why calibrate a laptop mic?	

Answers

Cover! 1: 10× · 2: 4000 Hz · 3: a spectrum: amount of each frequency · 4: low · 5: its readings are relative and gain-adjusted

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human FFT
2. Our acoustic lab
3. Record and measure level
4. FFT spectrum + spectrogram
5. Survey the school
6. Survey data
7. Acoustic scientist challenges

Class game

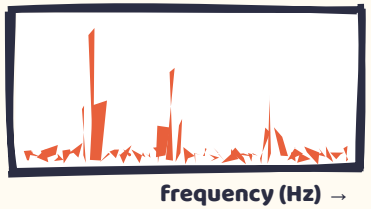
Human FFT

How to play

1. Three students hum together: one low, one middle, one high note (a 'chord').
2. The class closes eyes: can you pick out each note? That's what the FFT does mathematically.
3. Now one student stops. Who noticed which pitch disappeared?
4. Bonus: clap a rhythm. Is that a frequency too? (Yes: a very low one!)

Then discuss

Why is it easier for a computer than a human to separate 10 sounds at once?



Big idea

Complex sounds are sums of simple frequencies.

MAKE

Build it

Our acoustic lab

Python 3
numpy, matplotlib, sounddevice

Microphone
Laptop mic or USB mic

Phone sound meter app
For calibration

Tone generator app
Test the FFT with known Hz

School map
5 survey spots

Survey sheet
Spot, time, dB, sources

pip install numpy matplotlib sounddevice (on Linux you may also need PortAudio).

Protect hearing
Never play test tones loudly or into anyone's ear. Use headphones at low volume. Don't stand near very loud sources to measure them.

Objective 2 · Sample it · Build step 1 of 3

Record and measure level

```
import numpy as np, sounddevice as sd, matplotlib.pyplot as plt

FS, SECONDS, OFFSET = 44100, 3, 100        # OFFSET from calibration!
print("Recording...")
x = sd.rec(int(FS * SECONDS), samplerate=FS, channels=1, dtype="float32")
sd.wait()
x = x[:, 0]

rms = np.sqrt(np.mean(x ** 2))
dbfs = 20 * np.log10(rms + 1e-12)          # relative level
print(f"RMS {rms:.4f}  |  {dbfs:.1f} dBFS  |  approx {dbfs + OFFSET:.0f} dB")

t = np.arange(len(x)) / FS
plt.plot(t, x, linewidth=0.5)
plt.xlabel("time (s)"); plt.ylabel("amplitude"); plt.title("Waveform")
plt.show()
```

rec()

Records SECONDS × FS samples into a NumPy array.

RMS

Root mean square = typical size of the wave. Then dBFS = 20·log10(RMS).

Calibrate

Play steady noise; read a phone meter; set OFFSET = meter − dBFS.

MAKE

Objective 3 · FFT · Build step 2 of 3

FFT spectrum + spectrogram

```
# --- spectrum of the same recording ---
win = x * np.hanning(len(x))               # window reduces 'leakage'
spectrum = np.abs(np.fft.rfft(win))
freqs = np.fft.rfftfreq(len(win), 1 / FS)

mask = freqs > 20                          # ignore DC / rumble below 20 Hz
peak = freqs[mask][np.argmax(spectrum[mask])]
print(f"Dominant frequency: {peak:.0f} Hz")

top3 = freqs[mask][np.argsort(spectrum[mask])[-3:]][::-1]
print("Top peaks:", [f"{f:.0f} Hz" for f in top3])

fig, (a1, a2) = plt.subplots(2, 1, figsize=(10, 7))
a1.plot(freqs, 20 * np.log10(spectrum + 1e-9))
a1.set_xlim(20, 8000); a1.set_xscale("log")
a1.set_xlabel("Hz"); a1.set_ylabel("level (dB, relative)"); a1.set_title("Spectrum")
a2.specgram(x, Fs=FS, NFFT=2048, noverlap=1024)
a2.set_ylim(0, 8000); a2.set_xlabel("time (s)"); a2.set_ylabel("Hz"); a2.set_title("Spectrogram")
plt.tight_layout(); plt.show()
```

Test it

Play a 440 Hz tone from an app. Your peak should be ~440 Hz.

Log scales

Both ears and dB are logarithmic, so log axes show sound best.

Spectrogram

x = time, y = frequency, colour = loudness. Spot horns as bright bursts!

MAKE

Objective 4 · Build step 3 of 3

Survey the school

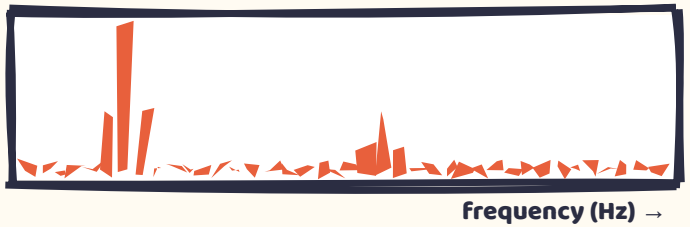
a Calibrate first: same sound, phone meter vs your program. Record your OFFSET.

b Visit 5 spots (gate, classroom, canteen, playground, near the road) at the same time of day.

c At each: 3 recordings of 10 s, laptop at 1 m height. Save level and top peaks.

d Name the main sources from the spectrogram + your ears (traffic, voices, generator…).

e Make a colour-coded map and one recommendation, with an honest uncertainty (±5 dB).



Check: A map with 5 spots, levels, sources and uncertainty.

Stay within school grounds with a teacher. Don't record conversations: measure levels, not words.

Test like an engineer

Survey data

Measurement	Value	Unit
Calibration offset		dB
Quietest spot + level		dB
Loudest spot + level		dB
440 Hz test tone peak		Hz
Main source at loudest spot		

Debug checklist

PortAudioError / no device? Check sd.query_devices() and pick the mic index.
Always −100 dBFS? Mic muted or blocked by OS privacy settings.
Peak at 50 Hz everywhere? Mains hum: check cables, or ignore below 60 Hz.

Report what you DON'T know too: mic limits, one day only, weather, time of day.

Level up!

Acoustic scientist challenges

Choose your challenge

Live meter: update level and spectrum every 0.5 s with an InputStream callback.

Classifier: label horn / voices / generator by their peak bands, then test on new clips.

ESP32 node: an I²S microphone (INMP441) on an ESP32 logging dB all day.

Before/after: measure a classroom with door open vs closed, curtains vs none.

Think like an engineer

Two sounds have the same dB but one feels much more annoying. Why might that be?

Boss level

Build a 'quiet-zone' alarm for the library that glows red above a calibrated level.

Objective check

Can I really do it? Prove it.

01	Why is 90 dB much more than 'a bit louder' than 80 dB?	○ ○ ○
02	What is your sample rate and the highest frequency you can capture?	○ ○ ○
03	Show your spectrum: what's the dominant frequency and what made it?	○ ○ ○
04	Show your survey map. What's your calibration offset and uncertainty?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Noise report: each team presents one finding and one recommendation to the principal, with evidence.

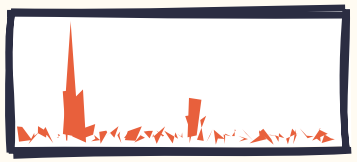
Show what you know

What did we learn?

- [x] Frequency = pitch, amplitude = size; dB is logarithmic
- [x] Sample rate ≥ 2 × highest frequency (Nyquist)
- [x] RMS → dBFS; calibrate for approximate dB
- [x] FFT turns time signals into spectra
- [x] Honest reports include uncertainty

At home

Use a phone meter app at home: find the loudest and quietest rooms.



Exit ticket: your FFT peak is at 120 Hz. Is that more likely a generator or a bird?

Next class:
Robotic Arm Inverse Kinematics!